

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/362092736>

CommentFinder: A Simpler, Faster, More Accurate Code Review Comments Recommendation

Conference Paper · July 2022

DOI: 10.1145/3540250.3549119

CITATIONS

72

READS

2,053

4 authors, including:



Yang Hong

Monash University (Australia)

564 PUBLICATIONS 37,639 CITATIONS

SEE PROFILE



Chakkrit Tantithamthavorn

Monash University (Australia)

144 PUBLICATIONS 6,027 CITATIONS

SEE PROFILE



Patanamon Thongtanunam

University of Melbourne

78 PUBLICATIONS 1,819 CITATIONS

SEE PROFILE

CommentFinder: A Simpler, Faster, More Accurate Code Review Comments Recommendation

Yang Hong
yang.hong1@monash.edu
Monash University
Australia

Patanamon Thongtanunam
patanamon.t@unimelb.edu.au
The University of Melbourne
Australia

Chakkrit Tantithamthavorn*
chakkrit@monash.edu
Monash University
Australia

Aldeida Aleti
aldeida.aleti@monash.edu
Monash University
Australia

ABSTRACT

Code review is an effective quality assurance practice, but can be labor-intensive since developers have to manually review the code and provide written feedback. Recently, a Deep Learning (DL)-based approach was introduced to automatically recommend code review comments based on changed methods. While the approach showed promising results, it requires expensive computational resources and time which limits its use in practice. To address this limitation, we propose COMMENTFINDER – a retrieval-based approach to recommend code review comments. Through an empirical evaluation of 151,019 changed methods, we evaluate the effectiveness and efficiency of COMMENTFINDER against the state-of-the-art approach. We find that when recommending the best-1 review comment candidate, our COMMENTFINDER is 32% better than prior work in recommending the correct code review comment. In addition, COMMENTFINDER is 49 times faster than the prior work. These findings highlight that our COMMENTFINDER could help reviewers reduce manual efforts by recommending code review comments, while requiring less computational time.

CCS CONCEPTS

• Software and its engineering;

KEYWORDS

Software Quality Assurance, Modern Code Review

ACM Reference Format:

Yang Hong, Chakkrit Tantithamthavorn, Patanamon Thongtanunam, and Aldeida Aleti. 2022. CommentFinder: A Simpler, Faster, More Accurate Code Review Comments Recommendation. In *Proceedings of the*

*The corresponding author

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ESEC/FSE '22, November 14–18, 2022, Singapore, Singapore

© 2022 Association for Computing Machinery.

ACM ISBN 978-1-4503-9413-0/22/11...\$15.00

<https://doi.org/10.1145/3540250.3549119>

30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '22), November 14–18, 2022, Singapore, Singapore. ACM, New York, NY, USA, 13 pages.
<https://doi.org/10.1145/3540250.3549119>

1 INTRODUCTION

Code review is a well-established quality assurance practice that is widely adopted in modern software development in both commercial and open-source projects [2, 6]. In the code review process, reviewers (i.e., developers other than the code author) manually examine the newly developed code and provide written feedback (called *code review comment*, henceforth, see Figure 1). The source code is then revised based on the feedback and integrated into the code repository once the reviewers approve it [2, 31]. Several studies have found that the code review process helps to improve the overall quality of software systems including reducing software defects [5, 36, 56] and improving software maintainability [6, 37, 49].

Due to the excessive reviewing effort required, performing code review becomes one of the top challenges faced by industry [4, 18, 19]. In particular, code review is labor-intensive since reviewers have to manually review the code and provide written feedback. In addition, to perform a scrutinized review, reviewers also need to spend time understanding the associated code and the purpose of the changes [2]. As projects are becoming more complex and large, the number of reviews is increasing over time (e.g., 3K reviews per month in Microsoft Bing [46] and 500 reviews per month in Linux [47]). Given this large number of reviews, prior research also found that the code authors may have to wait 15-64 hours unnecessarily at a time until they receive the code review comments from reviewers [22]. Hence, an approach that can automatically recommend code review comments for a given code change would be beneficial to save manual efforts in the code review process.

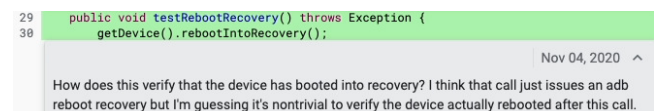


Figure 1: An example of a code review comment for a code snippet in the Android project.

With the goal of helping developers save the code review effort, a Deep Learning (DL)-based approach [61] is proposed to automatically generate code review comments based on a given changed

method. Particularly, Tufano *et al.* [61] formulated the code review comment recommendation as a Neural Machine Translation task (Code→Text) using a Text-To-Text Transfer Transformer (T5) architecture. Despite the promising results, the DL-based approach is complex and requires expensive computational time and resources. For example, Feng *et al.* built a pre-trained CodeBERT language model on the CodeSearchNet corpus which requires an NVIDIA DGX-2 with 16×NVIDIA Tesla V100 with 32GB memory with an approx. hardware cost of \$400K [12]. Unfortunately, the majority of software organizations may have limited access to such premium computing resources.

Recent studies [16, 20, 30, 33, 40] found that a simpler approach may achieve better performance than the DL-based approach with less computational time and resources. Importantly, Tufano *et al.* [61] do not evaluate their approach with any baseline approaches, raising an important concern if such the DL-based approach is the most accurate code review comment recommendation approach. Yet, little is known about whether a simpler approach can achieve higher accuracy than Tufano *et al.* [61] for the code review comment recommendation task. This central question is practically and significantly important for under-resourced software organizations, since practitioners may prefer simple over complex solutions [16].

In this paper, we propose COMMENTFINDER – a retrieval-based code review comment recommendation approach. Our intuition is that similar newly submitted code is likely to receive similar code review comments. Thus, our COMMENTFINDER is formulated as an information retrieval task. Given a corpus of code-comment pairs (i.e., a pair of a changed method and its associated human-written code review comment), COMMENTFINDER will use a given changed method as a query to search for the most similar changed methods in the code-comment corpus. Then, the associated code review comments are then used as the comment candidates for the given changed method. To expedite the search for the most similar changed methods, COMMENTFINDER performs the two-step similarity measurement. Specifically, COMMENTFINDER first measures a cosine distance between the given changed method and all changed methods in the corpus based on the Bag-of-Words model in order to select the ten nearest changed methods (i.e., lowest cosine distance). Then, COMMENTFINDER uses a text similarity metric called Gestalt Pattern Matching (GPM) to identify the k most similar changed methods based on the character-wise comparison.

Through an empirical evaluation of 151,019 changed methods that span across 11,289 large-scale open-source software projects that performed code reviews (i.e., 4,901 GitHub projects and 6,388 Gerrit projects), we address the following research questions:

(RQ1) How Effective is the COMMENTFINDER Compared to the State-of-the-art Approach?

Results. COMMENTFINDER can correctly recommend code review comments for 470 - 619 changed methods which are 32% - 51% higher than the Tufano *et al.* approach (i.e., 356 - 410 changed methods). In addition, with respect to the BLEU score (i.e., a measure to evaluate the similarity of the recommended code review comments against the ground truth), COMMENTFINDER achieves a mean BLEU score 3%-28% better than the state-of-the-art approach, indicating that COMMENTFINDER can recommend code review comments

that are more similar to the actual code review comments written by reviewers than the state-of-the-art approach.

(RQ2) How Efficient is the COMMENTFINDER Compared to the State-of-the-art Approach?

Results. COMMENTFINDER requires a total computational time of 4 minutes. The state-of-the-art approach, on the other hand, requires a total computational time of 194 minutes for training and recommending code review comments. This result shows that COMMENTFINDER is 49 times faster than the state-of-the-art approach, highlighting the efficiency of COMMENTFINDER.

(RQ3) What is the Impact of Similarity Techniques on the Effectiveness and Efficiency of COMMENTFINDER?

Results. When changing the distance metric from the cosine distance to the other two commonly-used metrics, the accuracy decreases by 4% - 12%, and the computational time increases by 50%. Similarly, when changing the text similarity metric from GPM to the other five commonly-used metrics, the computational time significantly increases (i.e., 425% - 25,750%) while the accuracy is slightly changed (i.e., 1% - 9%). These results suggest that while the distance metric can have a small impact on the effectiveness of COMMENTFINDER, the choice of the text similarity metrics plays an important role in the efficiency of COMMENTFINDER.

Significance & Contributions. These results lead us to conclude that COMMENTFINDER provides a substantial improvement for automatically recommending the code review comments in the context of code review. More specifically, our COMMENTFINDER is more effective (RQ1) and more efficient (RQ2) than the state-of-the-art approach [61], which suggests that our COMMENTFINDER could help developers save their efforts in a code review process better than the state-of-the-art approach. Based on our findings and insights in this work, we suggest that researchers should consider exploring simple solutions as it may achieve similar or better accuracy than a sophisticated DL-based approach, while requiring a less computational cost.

Novelty. This paper is the first to:

- Present COMMENTFINDER – A retrieval-based code review comment recommendation approach.
- Demonstrate that, for the code review comment recommendation task, the retrieval-based approach of COMMENTFINDER is more effective and efficient than the state-of-the-art DL-based approach.
- Demonstrate that, among the nine commonly-used similarity techniques, the use of cosine distance and GPM is the most cost-effective combination for our COMMENTFINDER.

Open Science. To support the open science initiative, we publish a replication package, which is publicly available on GitHub (<https://github.com/aws-sm-research/CommentFinder>).

Paper Organization. The paper is organized as follows. Section 2 describes background, motivation and limitation of the state-of-the-art approach. Section 3 presents our COMMENTFINDER. Section 4 describes our experimental setup. Section 5 presents the experimental results. Section 6 discusses the results of our COMMENTFINDER. Section 7 presents the related work. Section 8 discloses the threats to validity. Section 9 draws the conclusion.

2 BACKGROUND

In this section, we describe a code review process and situate the challenge based on the literature. We also briefly describe the state-of-the-art approach and its limitations.

2.1 Code Review Process

Code review is one of the well-established Software Quality Assurance (SQA) activities in modern collaborative software development practices [46, 49]. A code review process consists of five main steps (see Figure 2). Broadly speaking, in step ①, a code author will first submit a newly developed code (i.e., new code changes) to the code review tool (e.g., Gerrit, GitHub, or GitLab). Then, in step ②, to examine the newly developed code, the code author will invite reviewer(s) (i.e., the developers other than the code author) to review the newly developed code. In step ③, the reviewer(s) will inspect the code. If the reviewer(s) discover any issues in the submitted code, they will provide feedback (i.e., **code review comments**) to the specific areas of code. Figure 1 presents an example of a code review comment of a reviewer on the specific area of code in the Android project.¹ After that, in step ④, the reviewer(s) will make a decision on whether this submitted code can be merged into the code repository. In Step ⑤, if the reviewer(s) suggest that the submitted code is not ready to be merged and a revision is needed, the code author will revise the code to address the code review comments. Finally, if the submitted code has sufficient quality, the reviewer(s) will approve the submitted code to be integrated into the code repository.

2.2 Motivation & Usage Scenario

At the heart of the code review process is the code review comments. Such code review comments play an important role to guide developers to improve their submitted code [7]. Unfortunately, to identify issues and write code review comments, reviewers are required to take a considerable amount of time to understand the purpose and the context of the code [2]. Hong *et al.* [22] find that the code authors can wait for 15-64 hours to receive code review comments from reviewers. Such a long wait time may also stall the development process [10]. Hence, as projects become more complex and large with an increasing number of reviews over time, timely providing code review comments becomes one of the top challenges faced by development teams [7, 28].

Given the limited time and the large number of reviews, it is challenging for reviewers to thoroughly inspect the entire submitted code. Thus, an automated approach that can recommend code review comments for the submitted code would be beneficial to help developers save time. Figure 2 depicts a usage scenario of how our COMMENTFINDER could help reviewers to address this challenge. In step ③, instead of writing all new code review comments, our COMMENTFINDER recommends code review comments to the reviewers and code authors based on similar code artifacts that have received previous comments. Hence, the reviewers can effectively spend their efforts on reviewing other complex parts of the submitted code, which helps reduce the waiting time of code

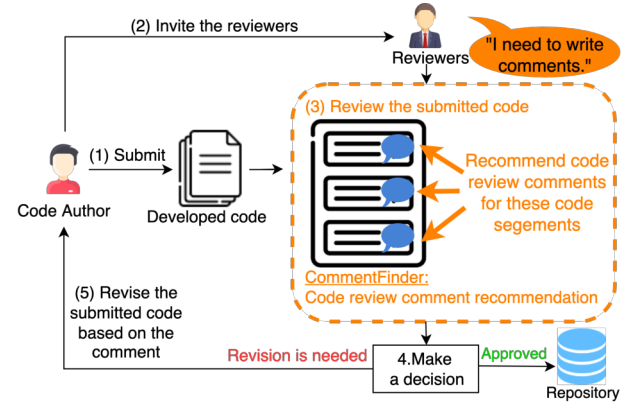


Figure 2: A usage scenario of COMMENTFINDER in the code review process.

authors for code review comments and shorten the overall time between each revision of submitted code.

2.3 The State-of-the-art Approach on Code Review Comment Recommendation

Tufano *et al.* [61] proposed a deep learning approach to automatically generate code review comments for a given *changed methods* (i.e., the changed or added methods in the submitted code). The approach by Tufano *et al.* employs a Neural Machine Translation (NMT) model to learn the mapping between an input sequence of the source code in a given changed method and an output sequence of a human-written code review comment associated with that changed method. To do so, Tufano *et al.* use a Text-To-Text Transfer Transformer (T5) architecture [43] to build a pre-trained language model based on a large text corpus from Stack Overflow and a large codebase from CodeSearchNet [23]. Then, they build a DL-based code review recommendation model by fine-tuning their pre-trained T5 model on their code review dataset (i.e., a corpus of code-comment pairs). Finally, they apply their fine-tuned T5 model on the testing dataset to generate code review comments.

Although the Tufano *et al.* approach achieves promising results, its deep learning approach is still expensive in terms of computational time and resources. Many studies [16, 20, 30, 33] found that simpler approaches may perform similar or better than the DL-based approaches for various software engineering tasks. For example, Hellendoorn *et al.* [20] found that a careful implementation of natural language process approaches like N-gram could outperform the deep learning approaches. Liu *et al.* [30] introduced a simple nearest-neighbor approach that achieves better performance than the state-of-the-art neural machine translation approach. Pornprasit and Tantithamthavorn [40] proposed a simple JITLine approach to identify the defective lines, which outperforms a DL-based approach CC2Vec [21]. Yet, little is known about whether a simpler approach can achieve higher accuracy than Tufano *et al.* [61] for the code review comment recommendation task.

¹<https://android-review.googlesource.com/c/device/google/cuttlefish/+1486302/2/tests/recovery/src/com/android/cuttlefish/tests/RecoveryTest.java>

3 COMMENTFINDER: A RETRIEVAL-BASED CODE REVIEW COMMENT RECOMMENDATION APPROACH

In this section, we present an overview of our COMMENTFINDER. The goal of our COMMENTFINDER is to recommend code review comments to the given changed methods. The underlying intuition in this paper is that similar submitted codes are likely to receive similar code review comments.

Overview. COMMENTFINDER leverages an Information Retrieval (IR) technique to recommend code review comments. Unlike deep learning techniques, IR techniques do not require the model training phase which will substantially save computational time. Similar to prior work [61], COMMENTFINDER takes a changed method in the testing data as input. This given changed method will be used as a query to search for similar changed methods in the training dataset. Specifically, the given changed method and the changed methods in the training data will be represented in a form of code token vectors based on the Bag-of-Words model. Then, COMMENTFINDER computes the distances between the vector of the given changed method and each of the vectors of the changed methods in the training dataset. The ten nearest changed methods will be retrieved. Since the Bag-of-Words model does not consider the order of the code tokens (i.e., considering only the frequencies), we use a text similarity metric called Gestalt Pattern Matching (GPM) to measure the similarity between the given changed method and the ten retrieved changed methods while taking the order of the code tokens into account. Then, the code review comments of the k most similar changed methods based on GPM will be recommended as code review comments for the given changed method. Figure 3 provides an overview of our approach which consists of three main steps. We describe the details of each step below.

(Step 1) Changed Method Vectorization. We represent all changed methods in the training dataset and test dataset as *vectors* using the Bag-of-Words model. To do so, we perform code tokenization to break each changed method into a sequence of code tokens. As suggested by Rahman *et al.* [44], we remove punctuation characters to ensure that the code tokens will not be artificially repetitive. After that, for each method, we convert the sequence of code tokens into the frequency vector of code tokens using the Countvectorize function of the Scikit-Learn Python library [39]. Since our studied programming language is case-sensitive, we neither perform lowercase nor techniques to reduce inflectional forms (i.e., stemming and lemmatization). Otherwise, the meaning of code tokens may be discarded. The vocabulary size is a common concern when using a Natural Language Processing (NLP) approach for predictions. To reduce the dimensions of our code token *vector*, we remove the code tokens that commonly appear across the methods (i.e., tokens that appear more than 50% of methods).

(Step 2) Changed Methods Retrieval. To retrieve the similar changed methods from the training dataset for a given changed method in the test dataset, we compute the cosine distance between the code token vector of the given method and each of the code token vectors of changed methods in the training dataset. The cosine distance is a mathematical model that estimates the lexical similarity between two text documents (i.e., changed methods in our context) [1, 30, 48]. Given the code token vectors of the changed

methods (i.e., M_{train} , M_{test}), the cosine distance using a calculation of $1 - \frac{M_{train} \cdot M_{test}}{\|M_{train}\| \|M_{test}\|}$. The lower cosine distance indicates that the two methods have a higher lexical similarity. In this work, we use the SequenceMatcher function implemented in the difflib Python library to compute the cosine distance between the changed methods. Then, the ten nearest methods (i.e., the lowest cosine distance) will be retrieved from all the changed methods in the training dataset.

(Step 3) Comment Recommendation. Based on our intuition, the code review comments associated with the retrieved changed method from Step 2 should be able to be recommended as code review comments for the given changed method in the testing data. However, the cosine distance model does not consider the order of the tokens. It is possible that the order of code tokens of retrieved changed methods may be different (which can be syntactically different) from the given method. Therefore, we use a text similarity approach called Gestalt Pattern Matching (GPM) to measure the text similarity and take the order of code tokens into account. Given a changed method and each of the ten retrieved methods, GPM first finds all the longest common substrings (LCSs) between the source code of the two methods. Then, it counts the total number of matching characters (i.e., N_{common}) in the common substrings. Then, the text similarity score is calculated using a formula of $\frac{2 \times N_{common}}{N_{changed} + N_{retrieved}}$, where $N_{changed}$ is the number of characters in the changed method and $N_{retrieved}$ is the number of characters in the similar method. Note that in this step, we use the actual source code of the methods without performing any preprocessing to reserve semantical and syntactical information of the code based on the GPM scores, the ten retrieved methods are then ranked. A changed method with the highest text similarity score is regarded as the most similar changed method for the given method in the test dataset. After that, the code review comments of the k most similar methods based on GPM will be used as the k recommended code review comments for the given method.

4 EXPERIMENTAL SETUP

In this section, we present the motivation for our research questions and describe the experimental dataset and our evaluation measures.

4.1 Research Questions

To evaluate our COMMENTFINDER, we formulate the following three research questions.

(RQ1) How Effective is the COMMENTFINDER Compared to the State-of-the-art Approach?

Motivation. In this work, we propose COMMENTFINDER to automatically recommend code review comments for given changed methods. Unlike the state-of-the-art approach [61] which leverages a deep learning technique, COMMENTFINDER leverages an IR technique to recommend code review comments by retrieving similar changed methods in the training dataset. Our underlying intuition is that similar submitted codes are likely to receive similar code review comments. Hence, we formulate this RQ to evaluate whether our COMMENTFINDER can recommend the code review comments that

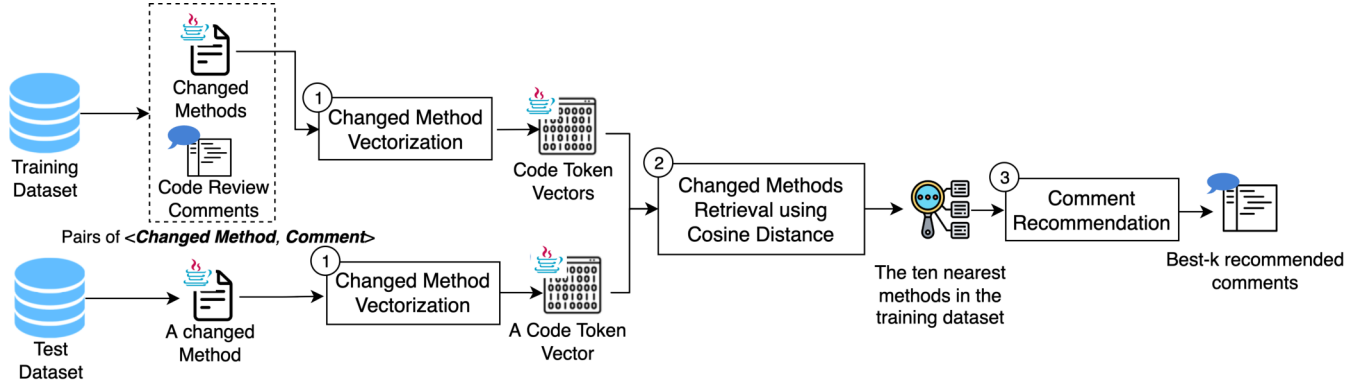


Figure 3: An overview of COMMENTFINDER for recommending the code review comments.

Table 1: The statistics of the experimental dataset obtained from the prior work [61].

#Pairs of Changed Methods and Code Review Comments		#Vocabulary Size	#Code Tokens			#Characters		
			1 st Q	Med	3 rd Q	1 st Q	Med	3 rd Q
151,019	Changed Methods	637,343	25	46	78	290	537	924
(Training:134,239; Test: 16,780)	Code Review Comments	-	7	13	23	43	80	142

are the same (or similar) as the actual code review comments written by reviewers.

(RQ2) How Efficient is the COMMENTFINDER Compared to the State-of-the-art Approach?

Motivation. Several studies raised concerns that deep learning techniques are often complex and expensive in terms of computational time [24, 33, 40]. For example, a prior study shows that the DL-based approach for automatically generating a commit message requires 38 hours for training the models [24]. Since the IR technique used in our COMMENTFINDER does not require model training, we hypothesize that this will substantially reduce the computational time. Thus, we formulate this RQ to evaluate the efficiency of COMMENTFINDER and the state-of-the-art approach [61] in terms of computational time.

(RQ3) What is the Impact of Similarity Techniques on the Effectiveness and Efficiency of COMMENTFINDER?

Motivation. To recommend code review comments, COMMENTFINDER leverages the cosine distance to retrieve the ten nearest changed methods and GPM to identify the k most similar changed methods. Although other distances and text similarity metrics are available, they may provide different results. Yet, little is known about the impact that these similarity techniques can have on the effectiveness and efficiency of our COMMENTFINDER. Hence, we formulate this RQ to further investigate the possible impact when the distance metrics or text similarity metrics in COMMENTFINDER are changed.

4.2 Experimental Dataset

In this work, we use the same training and testing datasets as used in the work of Tufano *et al.* [61] for a fair comparison between our COMMENTFINDER and the Tufano *et al.* approach. Table 1 describes the statistics of the dataset. The dataset consists of 151,019 changed methods from 4,901 GitHub projects and 6,388 Gerrit projects. For each changed method, it includes a pair of source code of a changed method and a code review comment. To minimize noises in the dataset, Tufano *et al.* have carefully curated and verified through a rigorous data cleansing and manual examination to exclude non-reviewer comments (i.e., written by the authors or automatically generated by bots), non-relevant comments (e.g., "looks good to me", "LGTM", "thank you"), and non-English comments. For changed methods in their dataset, at the median, the length of tokens and the length of characters in the changed method are 46 and 537. For code review comments in their dataset, at the median, the length of tokens and the length of characters in code review comments are 13 and 80. In addition, 7.5% of changed methods share the duplicate code review comments as other changed methods. We found that the majority of duplicate comments are a reserved programming language keyword (i.e., "final" for Java) that appears 133 times in the dataset.

4.3 Evaluation Measures

To evaluate the performance of our COMMENTFINDER and Tufano *et al.* approach, we measure the number of changed methods that achieve a perfect prediction (PP), i.e., the recommended comment exactly matches the ground truth (i.e., the actual code review comment provided by a reviewer). In our work, we use four different best- k candidates (i.e., $k = 1, 3, 5, 10$) for recommended code review comments. If one of k candidates exactly matches the ground truth,

we consider that our COMMENTFINDER achieves a perfect prediction, i.e., the comment is correctly recommended. For the given changed method, to quantify the magnitude of improvement of our COMMENTFINDER, we measure the percentage improvement of the perfect prediction using a calculation of $\frac{(PP_{our} - PP_{Tufano}) \times 100\%}{PP_{Tufano}}$, where PP_{our} is the number of perfect prediction achieved by our COMMENTFINDER and PP_{Tufano} is the number of perfect predictions achieved by our Tufano *et al.* approach.

Besides calculating the perfect prediction, we also compute the BLEU (Bilingual Evaluation Understudy) scores between our recommended code review comments and the ground truths [38]. BLEU evaluates how textually similar the recommended code review comment is to the ground truth. A BLEU score value of 1 indicates that the recommended comment is identical to the ground truth. In our work, we use the BLUE-4 variant (N-gram precision is 4). We also quantify the percentage improvement of BLEU score using the calculation of $\frac{(BLEU_{our} - BLEU_{Tufano}) \times 100\%}{BLEU_{Tufano}}$, where $BLEU_{our}$ is the mean of the BLEU score achieved by our COMMENTFINDER and $BLEU_{Tufano}$ is the mean of the BLEU score achieved by the Tufano *et al.* approach

5 EXPERIMENTAL RESULTS

In this section, we present approaches and results with respect to our three research questions.

(RQ1) How Effective is the COMMENTFINDER Compared to the State-of-the-art Approach?

Approach. In our RQ1, we evaluate how well our COMMENTFINDER can recommend the code review comments given changed methods, comparing against Tufano *et al.* approach [61]. In order to establish a fair comparison, we use the same training and test dataset as Tufano *et al.*’s work (see Table 1). Then, we use COMMENTFINDER to recommend code review comments for each of changed methods in the test dataset. For each recommendation, COMMENTFINDER provides best- k candidates (i.e., the code review comments of the k most similar methods in the training dataset based on GPM). Finally, we measure the accuracy of COMMENTFINDER and the Tufano *et al.*’s approach based on the two measures (i.e., Perfect prediction and BLEU score).

Result. Table 2 presents the results of COMMENTFINDER and the Tufano *et al.* approach for 16,780 changed methods. We measure the results based on four tiers of best- k candidates ($k = 1, 3, 5, 10$), where k is the number of recommended code review comments for a given changed method.

Our COMMENTFINDER achieves a perfect prediction 32%-51% higher than Tufano *et al.* approach. Table 2 shows the accuracy of our COMMENTFINDER and the Tufano *et al.* approach. We find that when the best- k candidate is 1, our COMMENTFINDER achieves the perfect prediction for 470 changed methods. The Tufano *et al.* approach achieves the perfect prediction for 356 changed methods. This result shows that our COMMENTFINDER achieves the perfect prediction 32% higher than the Tufano *et al.* approach. When the number of the best candidates is 3, 5, and 10, respectively, comparing with the Tufano *et al.* approach, the perfect predictions of

our COMMENTFINDER increase by 49%-51% (see Table 2). These results indicate that COMMENTFINDER substantially outperforms the Tufano *et al.* approach.

For the mean of BLEU score, our COMMENTFINDER achieves 3%-28% higher than the Tufano *et al.* approach. Table 2 shows that for the best-1 candidate, COMMENTFINDER achieves a mean BLEU score of 0.124, while the Tufano *et al.* approach achieves the mean BLUE score of 0.12. which is 3% lower than COMMENTFINDER. Even when we increase the number of best candidates to 3, 5, and 10, our COMMENTFINDER can achieve a higher BLEU score than the Tufano *et al.* approach by 22% for the best-3 candidates, 27% for the best-5 candidates, and 28% for the best-10 candidates. These results suggest that the quality of the code review comments recommended by COMMENTFINDER is better than the Tufano *et al.* approach.

Summary: COMMENTFINDER achieves perfect predictions 32% - 51% and BLEU scores 3% - 28% higher the Tufano *et al.* approach. These results suggest that COMMENTFINDER can effectively recommend code review comments that are the same or similar to the actual code review comments written by reviewers than the state-of-the-art approach.

(RQ2) How Efficient is the COMMENTFINDER Compared to the State-of-the-art Approach?

Approach. To address this RQ, we examine the computational time required by COMMENTFINDER and the Tufano *et al.* approach. To measure the computational time for COMMENTFINDER, we measure the time required to recommend code review comments for all changed methods in the test dataset. We run the experiment for COMMENTFINDER on a machine with AMD Ryzen 9 5950X CPU and 64GM of RAM. To ensure that our measurement is accurate, we reserve all resources for our experiment with no other tasks running. To measure the computational time of the Tufano *et al.* approach, we follow the implementation of the Tufano *et al.* approach and reproduce their experiment on Google Colaboratory.² Since the Tufano *et al.* approach requires Tensor Processing Units (TPU)³ to train the deep learning model. For the testing phase, we run the deep learning model of Tufano *et al.* on the same machine which also has an NVIDIA GeForce RTX 3090 for GPU. The computational time is measured based on recommending the best-1 candidate. Note that for the Tufano *et al.* approach, we use the same hyper-parameter setting (e.g., batch size, learning rate) as the prior work [61]. We also do not measure the computational time for building a pre-trained model in the Tufano *et al.* approach because this pre-trained model can be re-used.

Result. Our COMMENTFINDER is 49 times faster than the Tufano *et al.* approach for recommending the code review comments. Table 3 presents the computational time of our COMMENTFINDER and the Tufano *et al.* approach. As described in Section 3, COMMENTFINDER is based on the IR approach which does

²<https://research.google.com/colaboratory/faq.html>

³<https://colab.research.google.com/github/zaidalyafeai/Notebooks/blob/master/GPUvsTPU.ipynb#scrollTo=5moeHHv4shGw>

Table 2: The accuracy of our COMMENTFINDER comparing to Tufano et al. approach. The percentage value in the parenthesis indicates the percentage improvement of our COMMENTFINDER.

Approach	Best- k candidates = 1		Best- k candidates = 3		Best- k candidates = 5		Best- k candidates = 10	
	#PP	Mean of BLEU score	#PP	Mean of BLEU score	#PP	Mean of BLEU score	#PP	Mean of BLEU score
Our approach	470	0.124	566	0.183	605	0.209	619	0.238
Tufano <i>et al.</i> [61]	356	0.120	381	0.150	402	0.164	410	0.186
% Accuracy Improvement	32%	3%	49%	22%	50%	27%	51%	28%

Table 3: Computational Time of COMMENTFINDER and the Tufano et al. approach

Approach	Total Time	Training Phase		Testing Phase	
		Device	Training Time	Device	Testing Time
COMMENTFINDER	4 min	-	-	CPU	1 min (Cosine distance) + 3 min (GPM)
Tufano <i>et al.</i>	194 min	TPU	162 min	RTX 3090	32 min

not require a training phase. Hence, we do not have the computational time for training. On the other hand, Table 2 shows that the model training time of the Tufano *et al.* DL-based approach takes 162 minutes. In addition, for the testing phase, our approach takes 4 minutes to recommend code review comments for all the changed methods in the testing dataset, while the Tufano *et al.* approach takes 32 minutes. In total, our COMMENTFINDER required 4 minutes which is 49 times faster than the Tufano *et al.* approach with a total computational time of 194 minutes. These results indicate that COMMENTFINDER is more efficient than the Tufano *et al.* approach in terms of requiring less computational time.

Summary: COMMENTFINDER takes a total computational time of four minutes to recommend code review comments for all changed methods in the testing dataset, which is 49 times faster than the Tufano *et al.* approach. This result shows that COMMENTFINDER requires significantly less computation time, highlighting it is more efficient than the state-of-the-art approach.

(RQ3) What is the Impact of Similarity Techniques on the Effectiveness and Efficiency of COMMENTFINDER?

Approach. To address RQ3, we conduct two experiments to examine the perfect prediction and computational time when the distance metric used in Step 2 and the text similarity metric used in Step 3 of COMMENTFINDER are varied. Specifically, we vary the distance metrics from the cosine distance to the other two commonly-used distance metrics: (1) Euclidian distance and (2) Manhattan distance [9]. For the text similarity metrics, we vary from GPM to the other five commonly-used metrics: (1) Normalized Levenshtein (NL), (2) Metric Longest Common Subsequence (MLCS), (3) Damerau-Levenshtein (DL), (4) Levenshtein (L), and (5) Jaro-Winkler (JW) [3, 29, 65, 67, 69]. Note that when we analyze the impact of the distance metrics, we keep using the original text similarity measure (i.e. GPM). Therefore, we measure the accuracy and computational time for (1) Euclidean Distance + GPM and (2)

Manhattan Distance + GPM. Similarly, when we analyze the impact of the text similarity measure, we keep using the original distance metrics (i.e., cosine distance). Hence, we measure the accuracy and computational time for other five combinations: (1) Cosine distance + NL, (2) Cosine distance + MLCS, (3) Cosine distance + DL, (4) Cosine distance + L, and (5) Cosine distance + JW. The analysis results are based on the best-1 candidate.

Results. Table 4 presents the perfect predictions and computational time of our COMMENTFINDER (Cosine distance + GPM) and the seven additional combinations.

When using Euclidean or Manhattan distances instead of cosine distance, the perfect predictions decrease by 4% - 12%. Table 4 shows that comparing against our COMMENTFINDER, the perfect prediction decreases by 12% and 4% when using Euclidean distance and Manhattan distance instead of the cosine distance. We also observe that the computational time of these two combinations also increases (i.e., 50%). This result highlights that the accuracy and computational time are moderately impacted when the distance metric of COMMENTFINDER is changed to Euclidean or Manhattan distances.

When using the other five commonly-used text similarity metrics instead of GPM, the computational time increases by 425% - 25,750%. Table 4 shows that the computational time increases by 425% - 25,750% when the text similarity metric is changed from GPM to the other techniques. For example, when Damerau-Levenshtein (DL) is used, the computational time increases by 25,750%, and the perfect prediction also decreases by 6%. Although Table 4 shows that the use of Normalized Levenshtein (NL) could slightly increase the perfect prediction (i.e., 1% increase), this technique takes significantly computational time compared to GPM (i.e., 10,825% increase). Therefore, when considering both accuracy and computational time, GPM performs the best among the studied text similarity metrics. This result suggests that the choice of text similarity metrics can have a large impact on the computational time for our approach.

Table 4: The perfect prediction and computational time of the different combinations; and the percentage change compared to COMMENTFINDER.

	Combination	#PP	Computational Time (minutes)
	COMMENTFINDER (Cosine + GPM)	470	4
Distance	Euclidean + GPM	414 (-12%)	6 (+50%)
Metrics	Manhattan + GPM	450 (-4%)	6 (+50%)
	Cosine + NL	477 (+1%)	437 (+10,825%)
	Cosine + MLCS	464 (-1%)	405 (+10,025%)
Text	Cosine + DL	443 (-6%)	1034 (+25,750%)
Similarity	Cosine + L	443 (-6%)	438 (+10,850%)
	Cosine + JW	428 (-9%)	21 (+425%)

Summary: When the distance and text similarity metrics are changed to the other commonly-used metrics, the computational time significantly increases by 425% - 25,750%, while the accuracy is slightly changed (i.e., 1% - 9%). These results suggest that the choice of text similarity metrics plays an important role in the efficiency of COMMENTFINDER, and the use of cosine distance and GPM is the most cost-effective combination for COMMENTFINDER.

Table 5: The manual analysis of the code review comments recommended by our COMMENTFINDER and Tufano *et al.* approach based on the 100 sampled changed methods.

Category	COMMENTFINDER	Tufano
Perfect prediction	4	0
Semantically Equivalent	30	36
Alternative Solution	9	10
Incorrect	57	54

6 DISCUSSION

In this section, we discuss the experimental results, the contributions, and the limitation of our COMMENTFINDER.

6.1 Result Discussion

We now further discuss the performance of COMMENTFINDER based on the experimental results.

Why does COMMENTFINDER can effectively and efficiently recommend code review comments? Our RQ1 results highlight that we can recommend code review comments by retrieving similar changed methods. Figure 4 presents an example of the correctly recommended code review comment by COMMENTFINDER. The red text highlights the common substrings identified by GPM between the given method in the test dataset and the most similar method in the training dataset, confirming that both methods are very similar and receive the same concern. However, measuring the text similarity for the whole training dataset can be time-consuming. This highlights the benefit of using the cosine distance and the Bag-of-Words model to save computational time by retrieving a small set of nearest methods before measuring the character-wise text similarity of GPM. The results of RQ2 also show that the cosine distance takes only 1 minute to retrieve the ten nearest changed methods for each of the changed methods in the testing dataset. Lastly, our RQ3 also confirms that the cosine distance and GPM used by our COMMENTFINDER is more cost-effective than the other commonly-used distance metrics and text similarity metrics.

Are the code review comments that are incorrectly recommended by COMMENTFINDER still considered valuable for

the submitted code? Although our RQ1 has shown that COMMENTFINDER achieves perfect predictions (i.e., can correctly recommend code review comments) higher than the Tufano *et al.* approach, it is possible that the incorrect recommendations (i.e., non-perfect predictions) may also provide the comments that are semantically equivalent to the actual comments written by reviewers. Hence, similar to the prior work [61], we manually examine a sample of 100 changed methods to gain intuition on the number of valuable recommendations. Since we want to compare the manual result with the Tufano *et al.* approach, we use the same sample as used in the prior work, which is provided in their replication package [61].

We classify the code review comments recommended by COMMENTFINDER into one of the four categories: (1) *perfect prediction*, i.e., the recommended code review comment is exactly the same as the actual comment written by reviewers; (2) *semantically equivalent*, i.e., the recommended code review comment is semantically equivalent to the actual comment; (3) *alternative solution*, i.e., the recommended code review comment is not semantically equivalent to the actual comment but valuable for the submitted code; and (4) *incorrect*, i.e., the recommended code review comment is not meaningful for the submitted code.

Table 5 presents the results of our manual analysis on the recommendations of COMMENTFINDER. Although in most cases, the recommended code review comments of COMMENTFINDER are considered incorrect, it is worth noting that our COMMENTFINDER can recommend the correct code review comments (i.e., perfect prediction) for four changed methods while the Tufano *et al.* approach cannot. We also observe that 39% of the code review comments recommended by COMMENTFINDER are considered valuable (i.e., 30% for semantically equivalent and 9% for alternative solution).

A changed method from the test dataset	The best candidate from the training dataset
<pre>public String toString() { return "MistDataEvent with value: " + value.toString() + ",timestamp: " + timestamp; }</pre>	<pre>public String toString() { return "MistWatermarkEvent with timestamp: " + timestamp; }</pre>
Actual Code Review Comment: Please use StringBuilder	CommentFinder: Please use StringBuilder

Figure 4: An example of the *correctly recommended* code review comment by COMMENTFINDER.

<pre>public List getList(String key) { for(Config:child:children) { if(child.containsKey(key)) { return child.getList(key); } } return null; }</pre>
Actual Code Review Comment: call notFound instead of returning null
CommentFinder: call notFound

Figure 5: An example of the *semantically equivalent* code review comment recommended by COMMENTFINDER.

<pre>public void deactivate() { logger.debug("Deactivate"); }</pre>
Actual Code Review Comment: please remove this statement or enhance it to something more context/meaningful
CommentFinder: please decrease log level to trace or better remove statement

Figure 6: An example that recommended by COMMENTFINDER as *alternative solution* code review comment.

Figure 5 provides an example of the *semantically equivalent* code review comment that is recommended by COMMENTFINDER, showing that both actual and recommended comments have the same underline meaning or intention. Figure 6 provides an example of the *alternative solution* code review comment recommended by COMMENTFINDER, showing that the recommended comment is different but still relevant to the submitted code.

6.2 Contributions

Below, we discuss how our work will be beneficial to researchers and practitioners.

For researchers. Our results show that COMMENTFINDER which leverages an information retrieval technique performs better than the Tufano *et al.* approach which leverages a deep learning technique. Prior studies [16, 20, 30, 33] also show that a simple approach can outperform the DL-based approach in other software engineering tasks. For example, Liu *et al.* [30] proposed a *k*-nearest neighbor approach to automatically generate the commit message which

outperforms the neural machine translation approach. Fu *et al.* [16] and Majumder *et al.* [33] aim to link the related questions in a programmer discussion forum with simple approaches. Fu *et al.* [16] showed that their approach is 84 times faster than the deep learning approach. In addition, Majumder *et al.* [33] introduced an approach that has similar performance as the deep learning approach but it is over 500 times faster. Therefore, based on our results and prior studies, simple automated solutions should be first also be considered for software engineering tasks.

For practitioners. Our results highlight the potential of COMMENTFINDER to reduce human effort during the code review process by recommending code review comments. Moreover, compared with the deep learning approach, COMMENTFINDER requires much less computational time, which is often preferred by under-resourced software organizations [13]. In addition, as our COMMENTFINDER is a simple approach, it will be easier to implement and deploy, increasing the chance to be adopted by practitioners [16, 33]. Nevertheless, our COMMENTFINDER is designed for recommending a code review comment for a given changed method. The recommendation of code review comment for the finer granularity of source code (e.g., line level) may help developers better focus on the smaller scope of source code [22, 40, 64]. Hence, to increase the likelihood of adopting the approach in the practice, future work should further investigate a code review recommendation approach for the finer grain of source code.

It also is important to note that the ultimate goal of our work is to support developers on repetitive tasks, *not to fully automate the code review process*. Therefore, developers still play an important role in the code review process. In addition, at the current stage of the work, COMMENTFINDER would be suitable for software components that tolerate false positives as it is still a long way for research to achieve highly accurate code review comment recommendations.

6.3 Limitation

Although our results have highlighted that COMMENTFINDER is more effective and efficient than the Tufano *et al.* approach, the key limitation of COMMENTFINDER is that it cannot generate a new code review comment. In other words, COMMENTFINDER highly relies on the existing code review comments in the training dataset. If the submitted code is completely different from the submitted code in the training data, it is likely that COMMENTFINDER will not recommend correct code review comments. On the other hand, the deep learning approach can learn the patterns between the submitted code and code review comments. Thus, it can generate a new code review comment which may be valuable. Table 5 also

shows that the deep learning approach of Tufano *et al.* can recommend more code review comments that are considered semantically equivalent or alternative solutions than our approach. Therefore, to improve the accuracy, *future work should consider investigating approaches to leverage both retrieval-based and deep learning-based techniques for code review comment recommendation.*

7 RELATED WORK

In this section, we discuss the empirical studies related to code review to properly define the scope of our work and discuss the technological studies for automated code review to define the novelty of our work.

Code Review Comments. Prior studies found that code review comments from reviewers play a major role in the code review process in order to guide code authors to improve the quality of their submitted code [2, 28]. For example, Bacchelli *et al.* [2] found that the quality of code reviews is highly associated with code review comments. Many studies found that they play an important role in finding software defects [8, 34] or designing impactful changes in the source code [63]. However, it is common for reviewers to spend a substantial amount of effort in writing high-quality code review comments. The survey study of Kononenko *et al.* [28] also showed that providing code review feedback can be slow or not responsive due to the required amount of effort that reviewers have to spend.

Motivated by the findings of the prior work, this paper aims to address this challenge by proposing an automated approach that recommends code review comments for the repetitive parts of source code in order to reduce manual effort, enabling the reviewers to effectively spend time focusing on the more complex and critical parts of the software system.

AI for Code Review. Although modern code review is widely used in practice at various organizations, the code review practice is still manual and expensive. Thus, recent works leverage many Artificial Intelligence (AI) techniques to speed up the code review process. Importantly, prior studies proposed various recommendation approaches for the code review process. For example, code review prioritization approaches based on the characteristics of code changes [11, 32, 57] and the defect-proneness [14, 25–27, 35, 40–42, 45, 51–54, 64], reviewer recommendation approaches [4, 19, 55, 59, 66, 68], automated code transformation [15, 58, 60, 62].

Different from prior studies, this paper focuses on the code review comments recommendation task, which is currently rarely explored in software engineering, highlighting the significant contributions and novelty of our COMMENTFINDER.

Code Review Comments Recommendation. There exist a few approaches for code review comments recommendation. For example, Guo *et al.* [17] implemented a review-sharing approach to recommend the code review comments based on Convolutional Neural Network (CNN). Gupta *et al.* [18] presented a deep learning based-approach called DeepCodeReviewer that learns the relevance of a code snippet and recommends a code review comment from a repository of common comments. Siow *et al.* [50] proposed CORE, which is a deep learning-based code review comment recommendation approach that leverages a multi-level embedding and an attentional Long Short-Term Memory (LSTM) model. Later, Tufano *et al.* [61] leverages a modern and more advanced deep

learning architecture called T5 [43] to automatically recommend code review comments. Different from prior code review comments recommendation approaches, this paper is the first to propose a retrieval-based code review comments recommendation approach. Our results demonstrate that our COMMENTFINDER is more efficient and effective than the state-of-the-art approach, i.e., the Tufano *et al.* approach [61], highlighting that a simpler solution like using an IR technique can achieve higher accuracy than a sophisticated DL-based approach. Moreover, the simplicity of the approach is often preferred by practitioners, especially for under-resourced organization [16].

8 THREATS TO VALIDITY

Although we carefully design the experiment to ensure that our results are robust, there can be possible threats to the validity of our results and conclusions. Below, we discuss threats that may impact the results of our study.

Threats to the external validity relate to the degree to which our findings can be generalized to and across other projects and contexts. Our experiment is based on the large-scale dataset of 151,019 changed methods that span across a total of 11,289 software projects, which is provided by Tufano *et al.* [61]. Yet, these studied software projects mainly use the Java programming language. Hence, it is possible that our results may not be generalized to software projects that use other programming languages.

Threats to the construct validity relate to the granularity level of code changes, the quality of datasets, and the baseline selection. Our COMMENTFINDER is specifically designed to work at the method level for a fair comparison with the Tufano *et al.* approach which is also designed for recommending a code review comment for a given changed method [61]. The results of our COMMENTFINDER may be different for the different source code granularity (e.g., line level). Thus, the performance of our COMMENTFINDER may need to be revisited when the granularity of the source code is changed.

The quality of datasets may impact the results of our COMMENTFINDER. However, we note that Tufano *et al.* performed rigorous data cleaning and data filtering steps to mitigate the noise in the datasets (e.g., removing non-English code review comments, duplicated code review comments, bot comments). Yet, unknown noise may still creep into the studied datasets, which may impact the results of our COMMENTFINDER. Thus, future work should further discover unknown noise and explore how do they impact code review recommendation approaches.

While all these four approaches show efficiency in recommending the code review comments, we only select Tufano *et al.* 's approach as the baseline in our work due to the following reasons. On the one hand, Tufano *et al.* 's approach is based on a transformer which is known to be more efficient and effective than the techniques (i.e., CNN and LSTM) employed by the other three approaches. On the other hand, unlike the other three approaches, Tufano *et al.* 's work is publicly available. Thus, to

We select Tufano *et al.* [61] as a baseline comparison, instead of other code review recommendation approaches [17, 18, 50] due to the following reasons. First, Guo *et al.* [17] employed a CNN-based approach, and Siow *et al.* [50] and Gupta [18] leveraged an RNN-based NMT approach (e.g., Word Embedding and Long Short-Term

Memory), which are known to be less accurate than Transformer-based NMT approach used by Tufano *et al.* [61]. Second, the work of Tufano *et al.* [61] is publicly available, whereas the other three approaches [17, 18, 50] are not public. Selecting the publicly accessible approach as the baseline approach could mitigate the bias when reproducing the result of the baseline approach and increase the transparency for the experimental result comparison. Thus, Tufano *et al.* [61] is likely to be the most competitive baseline.

Threats to the internal validity relate to the text similarity component of our COMMENTFINDER. Our COMMENTFINDER employs the cosine distance and Gestalt Pattern Matching (GPM) techniques to retrieve and rank the similar changed methods in the training dataset for a given method in the testing dataset. However, other text similarity metrics may impact the recommendations of our COMMENTFINDER. Although our RQ3 examines this potential impact, our analysis is based on three commonly-used distance metrics and six commonly-used text similarity metrics. Finding an optimal text similarity metric can be very computationally expensive. Instead, the main goal of our work is not to find the best text similarity metric for our COMMENTFINDER, but to demonstrate that a retrieval-based approach can outperform the deep learning-based approach for the code review comment recommendation.

9 CONCLUSION

With the goal of helping developers save the code review effort, recent work proposed a Deep Learning (DL)-based approach [61] to recommend code review comments based on a given changed method. However, several studies raised concerns that the DL techniques are often complex and expensive in terms of computational time, hindering the adoption of the approach in the practice. Hence, in this paper, we propose COMMENTFINDER — a retrieval-based code review recommendation approach. Unlike the prior work, COMMENTFINDER leverages an Information Retrieval (IR) technique to recommend code review comments by retrieving similar changed methods. Our underlying intuition is that the similar submitted code is likely to receive similar comments. Through an empirical evaluation based on 151,019 changed methods spanned across 11,289 software projects, our results show that our COMMENTFINDER can accurately recommend code review comments with perfect predictions for 470 - 619 changed methods which are 32% - 51% higher than the state-of-the-art approach. Moreover, in terms of the total computational time, COMMENTFINDER is 49 times faster than the prior work. These results highlight that our COMMENTFINDER is more effective and efficient than the state-of-the-art approach. Based on the results, our COMMENTFINDER could potentially be beneficial to developers to reduce review effort by recommending code review comments for the repetitive parts of source code, enabling the reviewers to effectively spend time focusing on the more complex and critical parts.

Although our COMMENTFINDER provides a substantial improvement for the code review comment recommendation, we acknowledge that it is still a long way for researchers to develop a highly accurate and efficient code review comment recommendation approach. To increase the adoption in the practice, future work should explore approaches that recommend code review comments to any

kind of source code (even for the completely different one) with a finer granularity (e.g., line level) and with lower false positives.

ACKNOWLEDGEMENT

Chakkrit Tantithamthavorn was supported by the ARC's Discovery Early Career Researcher Award (DE200100941). Patanamom Thongtanunam was supported by the ARC's Discovery Early Career Researcher Award (DE210101091).

REFERENCES

- [1] Giuliano Antoniol, Gerardo Canfora, Gerardo Casazza, Andrea De Lucia, and Ettore Merlo. 2002. Recovering traceability links between code and documentation. *IEEE transactions on software engineering* 28, 10 (2002), 970–983.
- [2] Alberto Bacchelli and Christian Bird. 2013. Expectations, outcomes, and challenges of modern code review. In *Proceedings of the International Conference on Software Engineering (ICSE)*. IEEE Press, San Francisco, CA, USA, 712–721.
- [3] Daniel Bakkelund. 2009. An LCS-based string metric. *Oslo, Norway: University of Oslo* (2009).
- [4] Vipin Balachandran. 2013. Reducing human effort and improving quality in peer code reviews using automatic static analysis and reviewer recommendation. In *Proceedings of the International Conference on Software Engineering (ICSE)*. IEEE, San Francisco, CA, USA, 931–940.
- [5] Gabriele Bavota and Barbara Russo. 2015. Four eyes are better than two: On the impact of code reviews on software quality. In *Proceedings of IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, San Francisco, CA, USA, 81–90.
- [6] Moritz Beller, Alberto Bacchelli, Andy Zaidman, and Elmar Juergens. 2014. Modern code reviews in open-source projects: Which problems do they fix?. In *Proceedings of the working conference on mining software repositories (MSR)*. Association for Computing Machinery, New York, NY, USA, 202–211.
- [7] Amiangshu Bosu, Michaela Greiler, and Christian Bird. 2015. Characteristics of useful code reviews: An empirical study at microsoft. In *Proceedings of the IEEE/ACM Working Conference on Mining Software Repositories (MSR)*. IEEE, San Francisco, CA, USA, 146–156.
- [8] Chun Yong Chong, Patanamom Thongtanunam, and Chakkrit Tantithamthavorn. 2021. Assessing the Students Understanding and their Mistakes in Code Review Checklists—An Experience Report of 1,791 Code Review Checklists from 394 Students. In *Proceedings of the International Conference on Software Engineering: Joint Software Engineering Education and Training track (ICSE-JSEET)*. IEEE, San Francisco, CA, USA, 20–29.
- [9] Susan Craw. 2010. *Manhattan Distance*. Springer US, Boston, MA, 639–639. https://doi.org/10.1007/978-0-387-30164-8_506
- [10] Jacek Czerwinka, Michaela Greiler, and Jack Tilford. 2015. Code reviews do not find bugs. how the current code review best practice slows us down. In *Proceedings of the IEEE/ACM IEEE International Conference on Software Engineering (ICSE)*, Vol. 2. IEEE, San Francisco, CA, USA, 27–28.
- [11] Yuanrui Fan, Xin Xia, David Lo, and Shanjing Li. 2018. Early prediction of merged code changes to prioritize reviewing tasks. *Empirical Software Engineering* 23, 6 (2018), 3346–3393.
- [12] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, et al. 2020. Codebert: A pre-trained model for programming and natural languages. *arXiv preprint arXiv:2002.08155* (2020), 1536–1547.
- [13] Danyel Fisher, Rob DeLine, Mary Czerwinski, and Steven Drucker. 2012. Interactions with big data analytics. *Interactions* 19, 3 (2012), 50–59.
- [14] Michael Fu and Chakkrit Tantithamthavorn. 2022. LineVul: A Transformer-based Line-Level Vulnerability Prediction. In *2022 IEEE/ACM 19th International Conference on Mining Software Repositories (MSR)*. IEEE, San Francisco, CA, USA, 608–620.
- [15] Michael Fu, Chakkrit Tantithamthavorn, Trung Le, Van Nguyen, and Dinh Phung. 2022. VulRepair: A T5-Based Automated Software Vulnerability Repair. In *In Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE'22)*. To Appear.
- [16] Wei Fu and Tim Menzies. 2017. Easy over hard: A case study on deep learning. In *Proceedings of the Joint Meeting on Foundations of Software Engineering (FSE)*. Association for Computing Machinery, New York, NY, USA, 49–60.
- [17] Chenkai Guo, Hui Yang, Dengrong Huang, Jianwen Zhang, Naipeng Dong, Jing Xu, and Jingwen Zhu. 2020. Review sharing via deep semi-supervised code clone detection. *IEEE Access* 8 (2020), 24948–24965.
- [18] Anshul Gupta and Neel Sundaresan. 2018. Intelligent code reviews using deep learning. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD'18) Deep Learning Day*. Association for Computing Machinery, New York, NY, United States.

- [19] Christoph Hannebauer, Michael Patalas, Sebastian Stünkel, and Volker Gruhn. 2016. Automatically recommending code reviewers based on their expertise: An empirical comparison. In *Proceedings of the IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, San Francisco, CA, USA, 99–110.
- [20] Vincent J Hellendoorn and Premkumar Devanbu. 2017. Are deep neural networks the best choice for modeling source code?. In *Proceedings of the Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*. Association for Computing Machinery, New York, NY, USA, 763–773.
- [21] Thong Hoang, Hong Jin Kang, David Lo, and Julia Lawall. 2020. Cc2vec: Distributed representations of code changes. In *Proceedings of the ACM/IEEE International Conference on Software Engineering (ICSE)*. Association for Computing Machinery, New York, NY, USA, 518–529.
- [22] Yang Hong, Chakkrit Tantithamthavorn, and Patanamon Thongtanunam. 2022. Where Should I Look at? Recommending Lines that Reviewers Should Pay Attention To. In *Proceedings of IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, San Francisco, CA, USA, 1023–1034.
- [23] Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. 2019. Codesearchnet challenge: Evaluating the state of semantic code search. *arXiv preprint arXiv:1909.09436* (2019).
- [24] Siyuan Jiang, Ameer Armaly, and Collin McMillan. 2017. Automatically generating commit messages from diffs using neural machine translation. In *Proceedings of the International Conference on Automated Software Engineering (ASE)*. IEEE, San Francisco, CA, USA, 135–146.
- [25] Jirayus Jiarapakdee, Chakkrit Tantithamthavorn, Hoa Khanh Dam, and John Grundy. 2020. An Empirical Study of Model-Agnostic Techniques for Defect Prediction Models. *IEEE Transactions on Software Engineering (TSE)* 48, 1 (2020), 166–185.
- [26] Yasutaka Kamei, Emad Shihab, Bram Adams, Ahmed E Hassan, Audris Mockus, Anand Sinha, and Naoyasu Ubayashi. 2012. A large-scale empirical study of just-in-time quality assurance. *IEEE Transactions on Software Engineering* 39, 6 (2012), 757–773.
- [27] Chaiyakarn Khanan, Worawit Luewichana, Krissakorn Pruktharathikoon, Jirayus Jiarapakdee, Chakkrit Tantithamthavorn, Morakot Choetkiertikul, Chaiyong Ragkhitwetsagul, and Thanwadee Sunetnanta. 2020. JITBot: An Explainable Just-In-Time Defect Prediction Bot. In *2020 35th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, San Francisco, CA, USA, 1336–1339.
- [28] Oleksii Kononenko, Olga Baysal, and Michael W Godfrey. 2016. Code Review Quality: How Developers See It. In *Proceedings of the international conference on software engineering (ICSE)*. IEEE, San Francisco, CA, USA, 1028–1038.
- [29] Vladimir I Levenshtein et al. 1966. Binary codes capable of correcting deletions, insertions, and reversals. In *Soviet physics doklady*, Vol. 10. Soviet Union, 707–710.
- [30] Zhongxin Liu, Xin Xia, Ahmed E Hassan, David Lo, Zhenchang Xing, and Xinyu Wang. 2018. Neural-machine-translation-based commit message generation: how far are we?. In *Proceedings of the International Conference on Automated Software Engineering (ASE)*. Association for Computing Machinery, New York, NY, USA, 373–384.
- [31] Laura MacLeod, Michaela Greiler, Margaret-Anne Storey, Christian Bird, and Jacek Czerwona. 2018. Code Reviewing in the Trenches. *IEEE Software* 35 (2018), 34–42.
- [32] Chandra Maddila, Chetan Bansal, and Nachiappan Nagappan. 2019. Predicting pull request completion time: a case study on large scale cloud services. In *Proceedings of acm joint meeting on european software engineering conference and symposium on the foundations of software engineering (ESEC/FSE)*. Association for Computing Machinery, New York, NY, USA, 874–882.
- [33] Suveedep Majumder, Nikhila Balaji, Katie Brey, Wei Fu, and Tim Menzies. 2018. 500+ Times Faster than Deep Learning: A Case Study Exploring Faster Methods for Text Mining Stackoverflow. In *Proceedings of the International Conference on Mining Software Repositories (MSR) (MSR '18)*. Association for Computing Machinery, New York, NY, USA, 554–563.
- [34] Mika V Mäntylä and Casper Lassenius. 2008. What types of defects are really discovered in code reviews? *IEEE Transactions on Software Engineering* 35, 3 (2008), 430–448.
- [35] Shane McIntosh and Yasutaka Kamei. 2017. Are fix-inducing changes a moving target? a longitudinal case study of just-in-time defect prediction. *IEEE Transactions on Software Engineering (TSE)* 44 (2017), 412–428.
- [36] Shane McIntosh, Yasutaka Kamei, Bram Adams, and Ahmed E Hassan. 2014. The impact of code review coverage and code review participation on software quality: A case study of the qt, vtk, and itk projects. In *Proceedings of the Working Conference on Mining Software Repositories (MSR)*. Association for Computing Machinery, New York, NY, USA, 192–201.
- [37] Rodrigo Morales, Shane McIntosh, and Foutse Khomh. 2015. Do code review practices impact design quality? a case study of the qt, vtk, and itk projects. In *Proceedings of the international conference on software analysis, evolution, and reengineering (SANER)*. Association for Computing Machinery, New York, NY, USA, 171–180.
- [38] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the annual meeting of the Association for Computational Linguistics (ACL)*. Association for Computational Linguistics, USA, 311–318.
- [39] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. 2011. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830.
- [40] Chanathip Pornprasit and Chakkrit Tantithamthavorn. 2021. JITLine: A Simpler, Better, Faster, Finer-grained Just-In-Time Defect Prediction. *arXiv preprint arXiv:2103.07068* (2021), 369–379.
- [41] Chanathip Pornprasit and Chakkrit Tantithamthavorn. 2022. DeepLineDP: Towards a Deep Learning Approach for Line-Level Defect Prediction. *IEEE Transactions on Software Engineering* (2022), 1–1.
- [42] Chanathip Pornprasit, Chakkrit Tantithamthavorn, Jirayus Jiarapakdee, Michael Fu, and Patanamon Thongtanunam. 2021. PyExplainer: Explaining the Predictions of Just-In-Time Defect Models. In *proceedings of the IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, San Francisco, CA, USA, 407–418.
- [43] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. 2019. Exploring the limits of transfer learning with a unified text-to-text transformer. *arXiv preprint arXiv:1910.10683* 21, 1 (2019), 67 pages.
- [44] Musfiqur Rahman, Dharani Palani, and Peter C Rigby. 2019. Natural software revisited. In *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*. IEEE, San Francisco, CA, USA, 37–48.
- [45] Dilini Rajapaksha, Chakkrit Tantithamthavorn, Christoph Bergmeir, Wray Buntine, Jirayus Jiarapakdee, and John Grundy. 2021. SQAPLanner: Generating data-informed software quality improvement plans. *IEEE Transactions on Software Engineering* 48, 8 (2021), 2814–2835.
- [46] Peter C Rigby and Christian Bird. 2013. Convergent Contemporary Software Peer Review Practices. In *Proceedings of the European Software Engineering Conference and the International Symposium on the Foundations of Software Engineering (ESEC/FSE)*. Association for Computing Machinery, New York, NY, USA, 202–212.
- [47] Peter C Rigby, Daniel M German, Laura Cowen, and Margaret-Anne Storey. 2014. Peer review on open-source software projects: Parameters, statistical models, and theory. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 23, 4 (2014), 1–33.
- [48] S. Roopak and Tony Thomas. 2014. A Novel Phishing Page Detection Mechanism Using HTML Source Code Comparison and Cosine Similarity. In *proceedings of the International Conference on Advances in Computing and Communications (ICACC)*. 167–170. <https://doi.org/10.1109/ICACC.2014.47>
- [49] Caitlin Sadowski, Emma Söderberg, Luke Church, Michal Sipko, and Alberto Bacchelli. 2018. Modern Code Review: A Case Study at Google. In *Proceedings of ICSE (Companion)*. IEEE, San Francisco, CA, USA, 181–190.
- [50] Jing Kai Siow, Cuiyun Gao, Lingling Fan, Sen Chen, and Yang Liu. 2020. Core: Automating review recommendation for code changes. In *proceedings of the IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, San Francisco, CA, USA, 284–295.
- [51] Chakkrit Tantithamthavorn, Jirayus Jiarapakdee, and John Grundy. 2021. Actionable Analytics: Stop Telling Me What It Is; Please Tell Me What To Do. *IEEE Software* 38, 4 (2021), 115–120.
- [52] Chakkrit Tantithamthavorn, Shane McIntosh, Ahmed E Hassan, and Kenichi Matsumoto. 2016. Automated Parameter Optimization of Classification Techniques for Defect Prediction Models. In *Proceedings of the 38th International Conference on Software Engineering (ICSE)*. IEEE, San Francisco, CA, USA, 321–332.
- [53] Chakkrit Tantithamthavorn, Shane McIntosh, Ahmed E Hassan, and Kenichi Matsumoto. 2016. An empirical comparison of model validation techniques for defect prediction models. *IEEE Transactions on Software Engineering* 43, 1 (2016), 1–18.
- [54] Chakkrit Tantithamthavorn, Shane McIntosh, Ahmed E Hassan, and Kenichi Matsumoto. 2017. An Empirical Comparison of Model Validation Techniques for Defect Prediction Models. *IEEE Transactions on Software Engineering (TSE)* 43, 1 (2017), 1–18.
- [55] Patanamon Thongtanunam, Raula Gaikovina Kula, Ana Erika Camargo Cruz, Norihiro Yoshida, and Hajimu Iida. 2014. Improving code review effectiveness through reviewer recommendations. In *Proceedings of the International Workshop on Cooperative and Human Aspects of Software Engineering (CHASE)*. Association for Computing Machinery, New York, NY, USA, 119–122.
- [56] Patanamon Thongtanunam, Shane McIntosh, Ahmed E. Hassan, and Hajimu Iida. 2015. Investigating Code Review Practices in Defective Files: An Empirical Study of the Qt System. In *Proceedings of the IEEE/ACM Working Conference on Mining Software Repositories (MSR)*. IEEE, San Francisco, CA, USA, 168–179.
- [57] Patanamon Thongtanunam, Shane McIntosh, Ahmed E Hassan, and Hajimu Iida. 2018. Review participation in modern code review: An empirical study of the Android, Qt, and OpenStack projects (journal-first abstract). In *Proceedings of the International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, San Francisco, CA, USA, 475–475.

- [58] Patanamon Thongtanunam, Chanathip Pornprasit, and Chakkrit Tantithamthavorn. 2022. AutoTransform: Automated Code Transformation to Support Modern Code Review Process. (2022), 237–248.
- [59] Patanamon Thongtanunam, Chakkrit Tantithamthavorn, Raula Gaikovina Kula, Norihiro Yoshida, Hajimu Iida, and Ken-ichi Matsumoto. 2015. Who should review my code? a file location-based code-reviewer recommendation approach for modern code review. In *Proceedings of the IEEE International Conference on Software Analysis, Evolution, and Reengineering (SANER)*. IEEE, San Francisco, CA, USA, 141–150.
- [60] Michele Tufano, Jevgenija Pantiuchina, Cody Watson, Gabriele Bavota, and Denys Poshyvanyk. 2019. On learning meaningful code changes via neural machine translation. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, San Francisco, CA, USA, 25–36.
- [61] Rosalia Tufano, Simone Masiero, Antonio Mastropaolo, Luca Pascarella, Denys Poshyvanyk, and Gabriele Bavota. 2022. Using Pre-Trained Models to Boost Code Review Automation. *arXiv preprint arXiv:2201.06850* (2022), 2291–2302.
- [62] Rosalia Tufano, Luca Pascarella, Michele Tufano, Denys Poshyvanyk, and Gabriele Bavota. 2021. Towards Automating Code Review Activities. In *Proceedings of the International Conference on Software Engineering (ICSE)*. IEEE, San Francisco, CA, USA, 163–174.
- [63] Anderson Uchôa, Caio Barbosa, Daniel Coutinho, Willian Oizumi, Wesley KG Assunção, Silvia Regina Vergilio, Juliana Alves Pereira, Anderson Oliveira, and Alessandro Garcia. 2021. Predicting Design Impactful Changes in Modern Code Review: A Large-Scale Empirical Study. In *Proceedings of the IEEE/ACM International Conference on Mining Software Repositories (MSR)*. IEEE, San Francisco, CA, USA, 471–482.
- [64] Supatsara Wattanakriengkrai, Patanamon Thongtanunam, Chakkrit Tantithamthavorn, Hideaki Hata, and Kenichi Matsumoto. 2020. Predicting defective lines using a model-agnostic technique. *IEEE Transactions on Software Engineering (TSE)* 48, 5 (2020), 1480–1496.
- [65] William E Winkler. 1990. String comparator metrics and enhanced decision rules in the Fellegi-Sunter model of record linkage. (1990).
- [66] Yue Yu, Huaimin Wang, Gang Yin, and Charles X Ling. 2014. Reviewer recommender of pull-requests in GitHub. In *Proceedings of IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, San Francisco, CA, USA, 609–612.
- [67] Li Yujian and Liu Bo. 2007. A normalized Levenshtein distance metric. *IEEE transactions on pattern analysis and machine intelligence* 29, 6 (2007), 1091–1095.
- [68] Motahareh Bahrami Zanjani, Huzefa Kagdi, and Christian Bird. 2015. Automatically recommending peer reviewers in modern code review. *IEEE Transactions on Software Engineering* 42, 6 (2015), 530–543.
- [69] Chunchun Zhao and Sartaj Sahni. 2019. String correction using the Damerau-Levenshtein distance. *BMC bioinformatics* 20, 11 (2019), 1–28.